

GitFlame CodeRAG

Hybrid Issue-to-Evidence Retrieval for Repository Workflows

Sprint 2 Report: End-to-End Retrieval, Reranking, Experiments

GitFlame CodeRAG Team
Danil, Karim, Zhenya, Martin, Kirill

July 3, 2026

Abstract

GitFlame CodeRAG retrieves relevant code evidence for a GitHub/GitFlame-like issue, producing top- k evidence chunks (with file paths, line ranges, metadata and scores) suitable for downstream plan and code generation. Sprint 2 assembles the full pipeline: an issue is expanded into keywords and symbols, three retrievers (lexical BM25, dense code embeddings, and AST-structural candidates) generate candidates, their rankings are fused with Reciprocal Rank Fusion (RRF), and a cross-encoder reranker produces the final ranking. We evaluate the pipeline on a curated dataset of twelve synthetic repositories with 84 issues and known *expected files*, using Recall@K, MRR and MAP/NDCG, and quantify the contribution of each component through ablations and statistical significance tests. This report presents the dataset, method, experimental protocol and reproducibility setup; result tables and figures are wired to the evaluation outputs and are filled automatically as experiments complete.

1 Introduction

The GitFlame CodePilot assistant helps developers act on repository issues. This project focuses on the retrieval/RAG component for the *issue-to-plan* and later *plan-to-code* scenarios: given repository code files, a repository `.yaml` configuration and an issue, return the top- k code chunks that constitute the best evidence for resolving the issue. Recommendations and full code generation are out of scope; the target is *issue* $\rightarrow$ *relevant code evidence*. The central design principle is that raw source code is preserved as the source of truth (line numbers, real syntax), while separate normalized representations are used only for search (BM25 text, embedding text, keywords).

2 Related Work

Our pipeline combines three complementary retrieval signals. *Lexical* retrieval with BM25 ranks chunks by term overlap and remains a strong baseline for code search. *Dense* retrieval embeds queries and code chunks into a shared vector space with a code-aware embedding model and ranks by cosine similarity, capturing semantic matches that lexical methods miss. *Structural* retrieval uses AST-Grep to extract functions, handlers, tests and configuration nodes and to find candidates by symbols and structural patterns; it complements natural-language retrieval rather than replacing it. To combine rankers we use Reciprocal Rank Fusion, a simple and robust rank-aggregation method, followed by a cross-encoder *reranker* that jointly scores each query-chunk pair for a final ordering.

3 Dataset

Sprint 1 produced a curated, license-safe dataset of twelve small synthetic repositories spanning five languages. Each repository contains realistic source files (functions, classes, handlers, imports), a `repo.yml` configuration following the project template, and seven GitHub-like issues. Each issue provides `id`, `title`, `body`, `labels` and `expected_files`; `expected_chunks` are intentionally out of scope. The `expected_files` serve as ground truth for retrieval evaluation: for every issue we know which files contain the evidence a correct system should surface. All 130 expected files are verified to exist in the repositories and to survive the `analysis` include/exclude filtering of their config (Section 7).

Table 1: Per-repository overview of the evaluation dataset.

Repository	Language(s)	Files	Issues	Exp. files
repo_001_fastapi_blog	python	8	7	12
repo_002_go_url_shortener	go	6	7	10
repo_003_express_todo_api	javascript	6	7	11
repo_004_react_dashboard	typescript, javascript	6	7	11
repo_005_flask_auth_service	python	6	7	11
repo_006_go_grpc_inventory	go	6	7	12
repo_007_django_shop	python	7	7	11
repo_008_vue_notes_app	javascript, typescript	6	7	10
repo_009_nestjs_payments	typescript	6	7	10
repo_010_rust_cli_tool	rust	5	7	11
repo_011_python_etl_pipeline	python	6	7	9
repo_012_node_ts_graphql	typescript	6	7	12
Total (12)	–	74	84	130

Table 2: Repository count by declared language.

Language	Repositories
go	2
javascript	3
python	4
rust	1
typescript	4

4 Method

The end-to-end pipeline transforms an issue into ranked evidence chunks (Figure 1). Repository files are loaded, filtered by the `.yml` config and split into AST-aware chunks (with a fixed-window fallback); each chunk carries structural metadata, a BM25 text, an embedding text and keywords. At query time the issue is converted into a query text and keywords, the three retrievers produce candidate lists, RRF fuses them, and the reranker orders the fused candidates into the final top- k .

Dense similarity. Query and chunk embeddings are compared with cosine similarity,

$$\text{sim}(q, c) = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\| \|\mathbf{e}_c\|}. \quad (1)$$

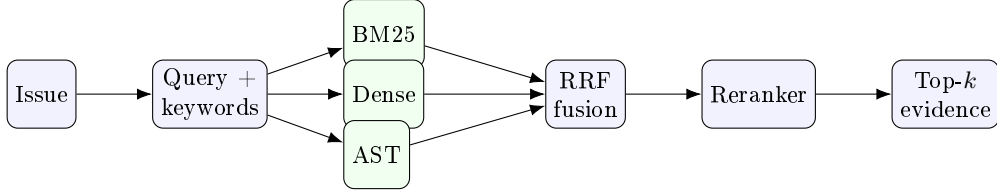


Figure 1: End-to-end retrieval pipeline: issue $\rightarrow$ BM25 / Dense / AST $\rightarrow$ RRF $\rightarrow$ reranker $\rightarrow$ final top- k evidence chunks.

Rank fusion. Given rankings $R_1, \dots, R_m$ from the retrievers, RRF scores each chunk c by

$$\text{RRF}(c) = \sum_{i=1}^m \frac{1}{k + \text{rank}_i(c)}, \quad (2)$$

where $\text{rank}_i(c)$ is the 1-based rank of c in R_i (absent chunks contribute 0) and k is a smoothing constant (`rrf_k`). The reranker then assigns a joint relevance score to each surviving query–chunk pair, and the final top- k evidence chunks are emitted in the shared evidence format. When the reranker model is unavailable, the pipeline falls back to the RRF ordering.

5 Experimental Setup

Ablations. We evaluate each signal and their combination: (i) BM25 only, (ii) Dense only, (iii) AST candidates only, (iv) RRF over all three, and (v) RRF + reranker. This isolates the contribution of lexical, semantic and structural evidence and of the reranking stage.

Metrics. Using `expected_files` as ground truth we report Recall@K, Mean Reciprocal Rank (MRR), Mean Average Precision (MAP) and NDCG@K for $K \in \{1, 3, 5, 10\}$, macro-averaged over the 84 issues.

Hyperparameters. Table 3 lists the search grid for the per-retriever candidate depths, the RRF constant, and the reranker/final cut-offs.

Table 3: Hyperparameter search grid.

Hyperparameter	Search values
<code>bm25_top_k</code>	50, 100
<code>dense_top_k</code>	50, 100
<code>ast_top_k</code>	20, 50
<code>rrf_k</code>	60
<code>reranker_top_k</code>	20, 50
<code>final_top_k</code>	5, 10

Significance testing. For the headline comparisons (e.g. RRF vs. each single retriever, and RRF + reranker vs. RRF) we test per-issue metric differences for significance and report the effect size and p -value.

6 Results

Table 4 reports the ablation metrics and Table 5 the significance tests; numbers are populated from the evaluation outputs (`reports/results/`). Figure 2 shows Recall@K curves per method.

Table 4: Ablation results (macro-averaged over 84 issues). *[filled from reports/results/metrics.json]*

Method	R@1	R@5	R@10	MRR	MAP	NDCG@10
BM25	—	—	—	—	—	—
Dense	—	—	—	—	—	—
AST candidates	—	—	—	—	—	—
RRF (BM25+Dense+AST)	—	—	—	—	—	—
RRF + Reranker	—	—	—	—	—	—

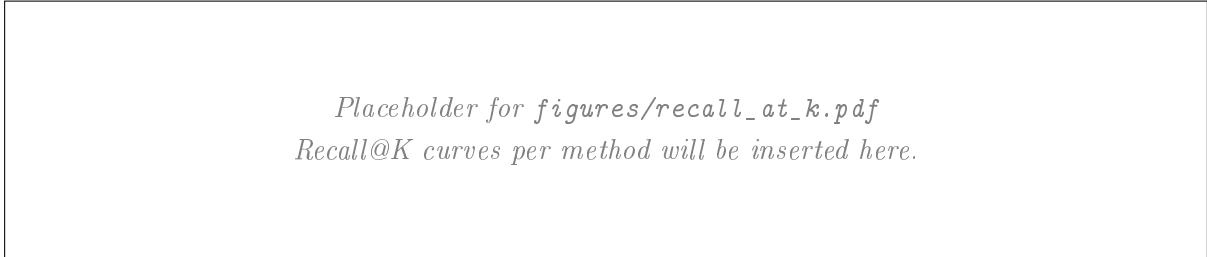


Figure 2: Recall@K per retrieval configuration.

[Discussion of which signals help which issue types, and whether the reranker gain over RRF is significant, to be written once results are in.]

7 Reproducibility

Every experiment input is described by a single generated manifest, `experiment_manifest.yml`, listing per repository its config, code and issues paths, revision, file and issue counts, and `expected_files` coverage, plus the experiment matrix and hyperparameter grid. The manifest is produced by `scripts/build_manifest.py` and validated by `validate_experiment_inputs()`, which reuses the ingestion functions to confirm that each `.yml` parses, code files load, issues are well-formed, and every `expected_file` exists and passes config filtering. On the current dataset all twelve repositories validate with zero errors. Report tables are regenerated from the manifest and results by `scripts/build_latex_tables.py`, so the document stays consistent with the data.

8 Conclusion and Future Work

Sprint 2 delivers the full issue-to-evidence pipeline (BM25/Dense/AST $\rightarrow$ RRF $\rightarrow$ reranker), a reproducible experiment manifest and validation, and a report scaffold wired to the evaluation outputs. Once the ablation and significance runs complete, the result tables and figures populate automatically. Future work includes richer issue-query construction, learned fusion weights, and extending the dataset toward `expected_chunks` for finer-grained evaluation.

References

- [1] S. Robertson and H. Zaragoza. *The Probabilistic Relevance Framework: BM25 and Beyond*. Foundations and Trends in Information Retrieval, 2009.
- [2] G. V. Cormack, C. L. A. Clarke, and S. Buettcher. *Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods*. SIGIR, 2009.

Table 5: Statistical significance of headline comparisons. *[filled from reports/results/significance.json]*

Comparison	Metric	Δ	p -value
RRF vs BM25	NDCG@10	–	–
RRF vs Dense	NDCG@10	–	–
RRF+Reranker vs RRF	NDCG@10	–	–

- [3] V. Karpukhin et al. *Dense Passage Retrieval for Open-Domain Question Answering*. EMNLP, 2020.